

# Document de Cadrage CynaSecure Marketplace+

Plateforme e-commerce de solutions de cybersécurité SaaS

SOC · EDR · XDR · CSPM · Zero Trust · NDR



---

ÉTUDIANTS

**Mouad BOUNOKRA · Pharell TITI  
Nolwen FERNANDES · Wissal LIDAM**

ÉTABLISSEMENT

**H3 HITEMA · Promotion 2025-2026**

TUTEUR PÉDAGOGIQUE

**M. Guillaume Manitra**

TYPE

**Projet fil rouge B3 · CPI**

# Table des matières

## **Partie I - Spécifications fonctionnelles**

1. Introduction fonctionnelle
2. Sitemap de la plateforme
3. Description détaillée des pages

## **Partie II - Architecture cible du système**

1. Diagramme d'architecture globale
2. Architecture backend (Symfony)
3. Flux de données
4. Modèle de données (ERD)

## **Partie III - Choix technologiques**

1. Technologies frontend
2. Technologies backend
3. Base de données
4. Sécurité et paiement
5. Application mobile

## **Partie IV - Méthodologie, organisation & planning**

1. Approche Agile Scrum
2. Organisation de l'équipe
3. Outils de développement
4. Planning du projet (Gantt)

# Partie I - Spécifications fonctionnelles

## 1. Introduction fonctionnelle

CynaSecure Marketplace+ est une plateforme e-commerce destinée à la commercialisation de solutions de cybersécurité proposées en mode SaaS. Elle permet à des entreprises de parcourir un catalogue de services spécialisés (SOC managé, EDR, XDR, sécurité du cloud, gestion des identités), de comparer les offres et de souscrire en ligne à celles qui correspondent à leurs besoins.

Le projet vise une clientèle professionnelle (B2B). L'enjeu est de rendre l'achat de services de sécurité aussi simple qu'un achat e-commerce classique, tout en gardant le niveau d'exigence attendu sur ce type de produit : clarté de l'offre, sécurité des transactions et suivi des abonnements.

Concrètement, la plateforme permet :

- de découvrir et de comparer les différents services de cybersécurité du catalogue ;
- de souscrire un abonnement mensuel ou annuel et de payer en ligne de façon sécurisée ;
- de gérer ses abonnements, ses commandes et ses moyens de paiement depuis un espace personnel ;
- aux administrateurs de Cyna, de piloter le catalogue, les utilisateurs, les commandes et le contenu du site via un back-office dédié.

Le périmètre couvre trois livrables complémentaires : un site web (partie publique et back-office), une API REST qui centralise toute la logique métier, et une application mobile qui reprend les mêmes parcours que le site.

## 2. Sitemap de la plateforme

Le sitemap ci-dessous présente l'organisation de la navigation. Elle a été pensée pour rester simple : un visiteur doit pouvoir passer du catalogue au paiement en quelques clics, et un client retrouver ses abonnements sans chercher.

```
Accueil
├── Catalogue
│   ├── Recherche avancée (texte, prix, catégorie, disponibilité, type)
│   └── Fiche service (détails, tarifs, services similaires)
├── Panier → Tunnel de commande (identité → adresse → paiement → confirmation)
├── Authentification
│   ├── Connexion (option « Se souvenir de moi »)
│   ├── Inscription (validation par e-mail)
│   ├── Mot de passe oublié
│   └── Double authentification (2FA / TOTP)
├── Espace utilisateur
│   ├── Profil · Carnet d'adresses · Moyens de paiement
│   ├── Mes abonnements · Mes commandes (factures PDF)
│   └── Tableau de bord
```

- Contact (formulaire + chatbot)
- Pages légales (CGU, mentions, confidentialité, à propos)
- Back-office (réservé administrateurs, 2FA obligatoire)
  - Tableau de bord (KPI + graphiques)
  - Services · Catégories · Utilisateurs
  - Abonnements · Paiements · Codes promo
  - Contenu de la page d'accueil · Messages de contact

### 3. Description détaillée des pages

Cette section décrit, page par page, l'objectif visé, le contenu affiché, les actions possibles et les cas particuliers à gérer. Le site est conçu en mobile-first et conserve une charte sombre cohérente sur l'ensemble des écrans.

#### 3.1. Page d'accueil

**Objectif :** Présenter immédiatement la valeur de la plateforme, rassurer par des indicateurs concrets et orienter le visiteur vers le catalogue ou la création de compte.

##### Contenu affiché

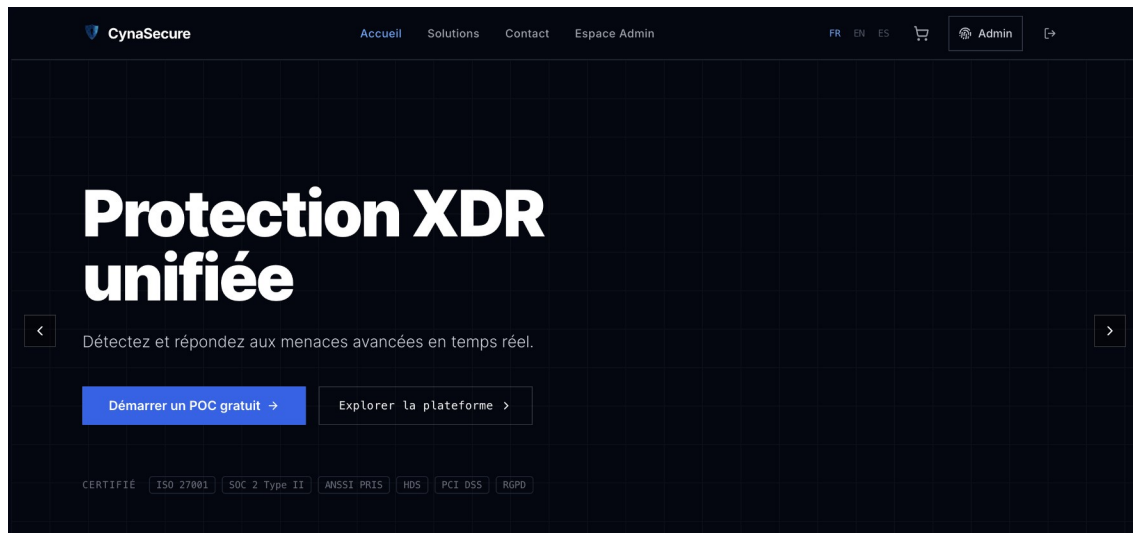
- En-tête fixe : logo, liens Accueil / Catalogue / Contact, icône panier avec compteur, bouton Connexion, sélecteur de langue (FR/EN/ES).
- Section hero : titre, sous-titre présentant les services SOC, EDR, XDR, et deux appels à l'action.
- Carrousel éditorial dont le contenu est géré depuis le back-office.
- Bandeau de réassurance : 500+ entreprises protégées, 99,9 % de disponibilité SLA, SOC 24/7, MTTR < 15 minutes.
- Grille des catégories de services, avec image, description courte et nombre de services.
- Section « Top produits du moment », sélectionnée par les administrateurs.
- Pied de page : liens Solutions / Entreprise / Légal et réseaux sociaux.

##### Actions utilisateur

- Explorer le catalogue
- Ouvrir une catégorie
- Découvrir un service
- Ajouter un service au panier
- Se connecter ou créer un compte
- Changer de langue

##### Cas particuliers

- Utilisateur connecté : un avatar remplace le bouton Connexion.
- Panier non vide : un badge numérique apparaît sur l'icône panier.
- Catégorie sans service : la tuile n'est pas affichée.
- Mise en page entièrement responsive (mobile-first).



Capture 1 - Page d'accueil

### 3.2. Page Catalogue

**Objectif :** Centraliser toutes les solutions et permettre de trouver rapidement la bonne via la recherche, les filtres et le tri.

#### Contenu affiché

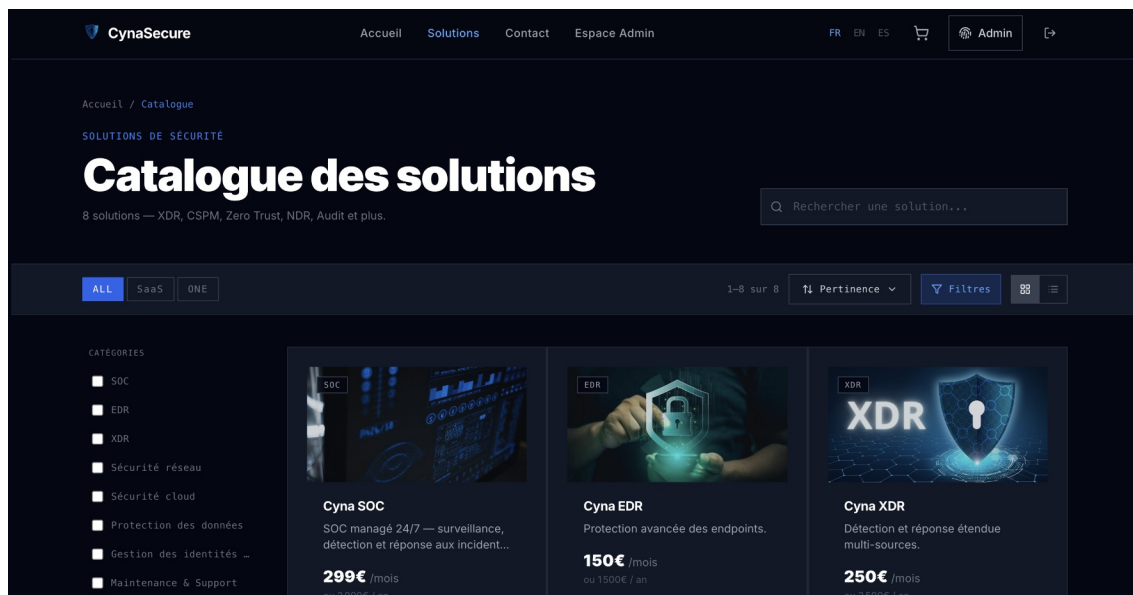
- Barre de recherche avec filtrage en temps réel (déclenché côté serveur).
- Recherche avancée à cinq facettes : texte, fourchette de prix, catégorie (multi-sélection), disponibilité, type (abonnement / achat unique).
- Tri : pertinence, prix croissant, prix décroissant, nouveauté.
- Grille de services : nom, description courte, prix mensuel, badge, bouton Découvrir.
- Pagination côté serveur et filtres synchronisés dans l'URL.

#### Actions utilisateur

- Rechercher un service
- Filtrer par catégorie et par prix
- Trier les résultats
- Découvrir un service
- Ajouter au panier

#### Cas particuliers

- Aucun résultat : message dédié et bouton de réinitialisation.
- Service déjà au panier : l'icône est remplacée par une coche.
- Sur mobile : grille en une colonne, filtres dans un panneau dédié.



Capture 2 - Catalogue et recherche avancée

### 3.3. Fiche service

**Objectif :** Fournir toutes les informations pour évaluer un service et faciliter la souscription. C'est la principale page de conversion.

#### Contenu affiché

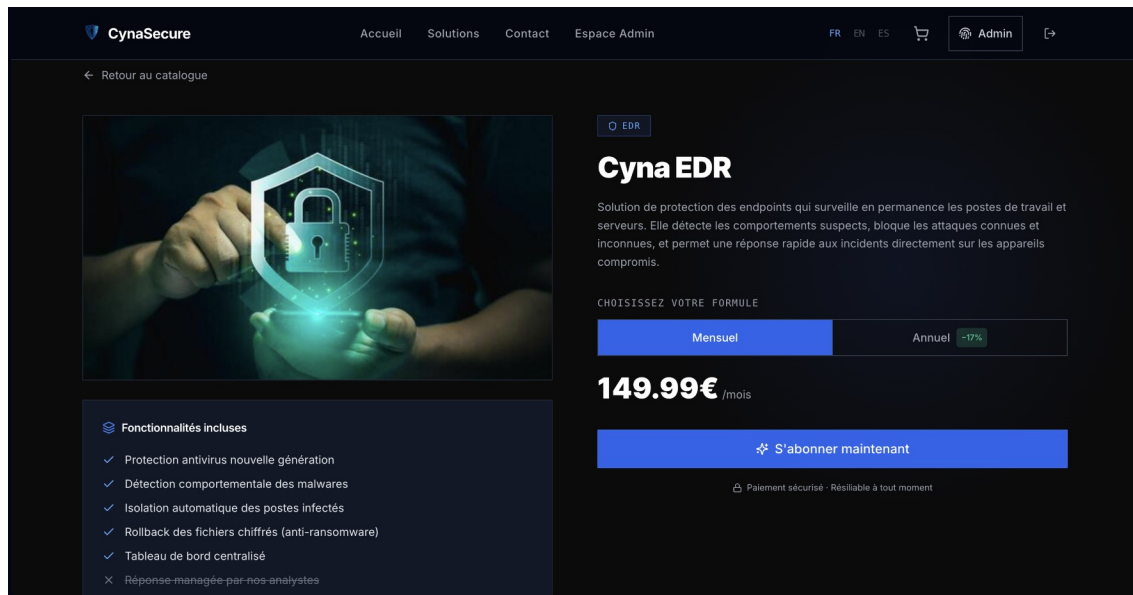
- Fil d'Ariane et badge de catégorie.
- Carrousel d'illustrations du service.
- Description détaillée et liste des fonctionnalités incluses (à cocher).
- Section dédiée aux caractéristiques techniques (SLA, MTTD, conformité, hébergement, support).
- Panneau de tarification avec bascule mensuel / annuel et économie annuelle affichée.
- Liste de six services similaires de la même catégorie.

#### Actions utilisateur

- Basculer entre mensuel et annuel
- S'abonner ou essayer
- Ajouter au panier
- Consulter un service similaire

#### Cas particuliers

- Service indisponible : le bouton devient « Service indisponible » et est désactivé.
- Service déjà au panier : le bouton d'ajout est désactivé.
- Sur mobile : le panneau de tarification passe en bas de page.



Capture 3 - Fiche service

### 3.4. Panier

**Objectif :** Récapituler les services sélectionnés et préparer le passage en commande, sans imposer la connexion à ce stade.

#### Contenu affiché

- Liste des services avec leur cycle de facturation et leur prix unitaire.
- Bascule mensuel / annuel par ligne, total recalculé en temps réel.
- Champ de code promotionnel avec validation côté serveur.
- Récapitulatif HT, TVA 20 %, total TTC.
- Bouton « Passer à la caisse ».

#### Actions utilisateur

- Modifier le cycle d'un service
- Appliquer ou retirer un code promo
- Supprimer un service
- Vider le panier
- Passer commande

#### Cas particuliers

- Panier vide : message dédié et lien vers le catalogue.
- Service devenu indisponible : alerte et possibilité de le retirer.
- Persistance du panier durant la session.

### 3.5. Tunnel de commande (checkout)

**Objectif :** Finaliser la commande avec un parcours clair et sécurisé, accessible aussi aux invités.

#### Contenu affiché

- Indicateur d'étapes : identité → adresse → paiement → confirmation.
- Étape identité : connexion, inscription, ou poursuite en tant qu'invité (e-mail requis).

- Étape adresse : formulaire complet ou sélection d'une adresse enregistrée.
- Étape paiement : choix entre carte bancaire (Stripe) et PayPal.
- Étape confirmation : récapitulatif, numéro de facture, téléchargement du PDF, e-mail de confirmation.

#### Actions utilisateur

- Renseigner l'adresse de facturation
- Choisir le moyen de paiement
- Confirmer et payer
- Télécharger la facture
- Revenir au panier

#### Cas particuliers

- Paiement échoué : message d'erreur explicite, possibilité de réessayer.
- Panier vide : redirection vers le catalogue.
- Montant systématiquement recalculé côté serveur avant paiement.

Capture 4 - Tunnel de commande

### 3.6. Connexion / Inscription

**Objectif :** Permettre une authentification rapide et sécurisée.

#### Contenu affiché

- Carte centrale avec onglets Connexion / Inscription.
- Formulaire d'inscription avec contrôle de robustesse du mot de passe en temps réel.
- Inscription confirmée par un e-mail de validation (lien valable 24 h).
- Double authentification (TOTP) proposée et obligatoire pour les administrateurs.

#### Actions utilisateur

- Se connecter
- Créer un compte
- Afficher / masquer le mot de passe
- Demander la réinitialisation du mot de passe



- Saisir un code 2FA

#### **Cas particuliers**

- Identifiants incorrects : message d'erreur.
- Compte non vérifié : invitation à confirmer l'e-mail.
- Déjà connecté : redirection vers l'espace personnel.

### **3.7. Espace utilisateur**

**Objectif :** Centraliser la gestion du compte, des abonnements et des commandes.

#### **Contenu affiché**

- Tableau de bord : abonnements actifs et dernières commandes.
- Profil, changement d'e-mail (validé par lien) et de mot de passe.
- Carnet d'adresses et moyens de paiement enregistrés.
- Mes abonnements : renouveler, changer de cycle, résilier.
- Mes commandes : historique groupé par année avec téléchargement des factures PDF.

#### **Actions utilisateur**

- Modifier son profil
- Gérer ses adresses et cartes
- Gérer ses abonnements
- Télécharger une facture
- Activer la double authentification

#### **Cas particuliers**

- Action sensible : confirmation par mot de passe ou par lien e-mail.
- Aucun abonnement : message et lien vers le catalogue.
- Notification de succès après chaque modification.

### **3.8. Back-office administrateur**

**Objectif :** Offrir aux administrateurs une interface complète de pilotage, réservée et protégée par 2FA.

#### **Contenu affiché**

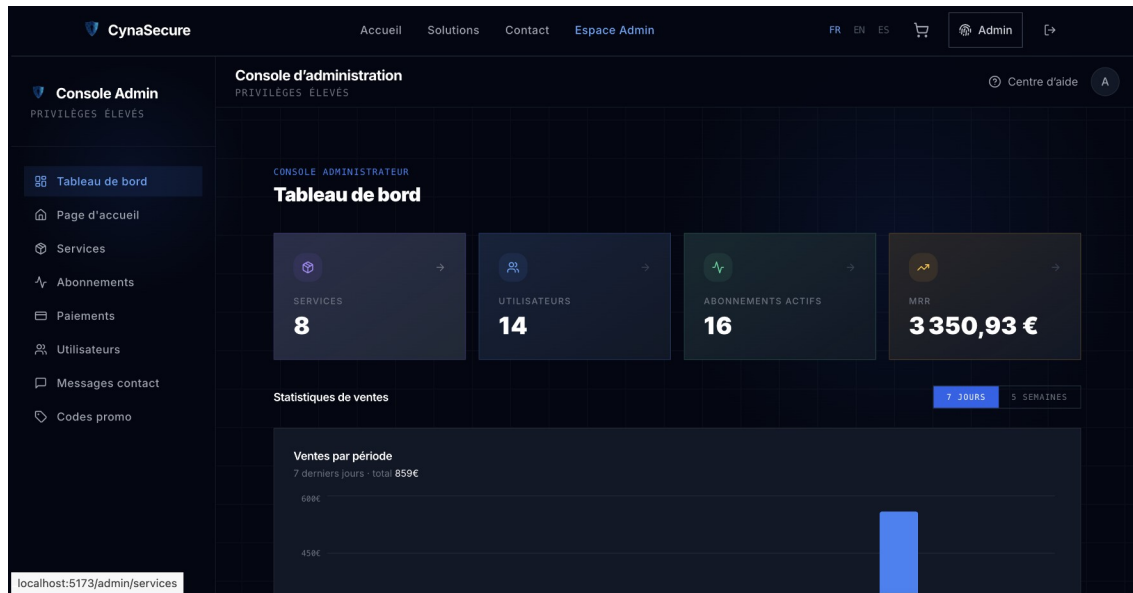
- Tableau de bord : indicateurs clés (services, utilisateurs, abonnements actifs, MRR) et trois graphiques (ventes par jour/semaine, paniers moyens par catégorie, répartition par catégorie).
- Gestion des services, catégories, utilisateurs, abonnements et paiements.
- Gestion des codes promotionnels et du contenu de la page d'accueil.
- Consultation des messages de contact et des conversations du chatbot.
- Listes paginées, triables, avec filtres synchronisés dans l'URL.

#### **Actions utilisateur**

- Créer, modifier ou désactiver un service
- Gérer les rôles et les comptes
- Suivre les commandes et les paiements
- Créer un code promo
- Éditer la page d'accueil

#### **Cas particuliers**

- Accès refusé aux non-administrateurs et redirection.
- Double authentification exigée avant tout accès au back-office.
- Confirmation demandée avant les actions destructrices.



Capture 5 - Tableau de bord administrateur

## 4. Contraintes, livrables et critères de réussite

Au-delà des fonctionnalités, le projet est encadré par des contraintes et des livrables attendus qui structurent le travail de l'équipe.

### Contraintes du projet

**Techniques :** API REST sécurisée, base relationnelle, paiement conforme PCI-DSS, application responsive et mobile, conformité RGPD et accessibilité WCAG 2.1.

**Organisationnelles :** équipe réduite, méthode Agile imposée, deux dépôts Git distincts (web et mobile), documentation technique et soutenance orale à produire.

**Délais :** projet conduit d'octobre 2025 à mai 2026, rythmé par les jalons du fil rouge (cadrage, rendu écrit, soutenance, livrable final).

### Livrables attendus

- Le code source du site web et du back-office (dépôt Git).
- Le code source de l'application mobile (dépôt Git distinct).
- Une application web responsive de type Single Page Application.
- La documentation technique : guide d'installation, documentation de l'API (Swagger), structure du code.
- Le document de cadrage, le document d'architecture technique (DAT) et les diagrammes associés.
- Les tests automatisés et le rapport d'accessibilité.

### Critères de réussite

- Le parcours d'achat complet fonctionne, du catalogue à la facture, sur web et mobile.
- Les paiements aboutissent en environnement de test (Stripe et PayPal).

- Le back-office permet de gérer l'ensemble des ressources sans intervention technique.
- La sécurité est effective : 2FA, mots de passe forts, vérification e-mail, webhooks signés.
- La documentation et les diagrammes permettent à un tiers de comprendre et reprendre le projet.

## Gestion des risques

| Risque                                 | Impact | Mesure d'atténuation                               |
|----------------------------------------|--------|----------------------------------------------------|
| Retard sur une fonctionnalité complexe | Moyen  | Découpage en sprints, priorisation du backlog      |
| Fuite de clés API / secrets            | Élevé  | Variables d'environnement, fichiers non versionnés |
| Païement : incohérence de statut       | Élevé  | Webhooks signés + traitement idempotent            |
| Indisponibilité d'un service externe   | Moyen  | Mode test (sandbox), gestion des erreurs réseau    |
| Perte de code / régression             | Moyen  | Dépôt Git, revues de code, tests automatisés       |

## Partie II - Architecture cible du système

### 1. Diagramme d'architecture globale

L'architecture repose sur une séparation nette entre les clients (le site web et l'application mobile) et le serveur (une API REST développée avec Symfony). Les deux clients consomment exactement les mêmes endpoints : il n'y a donc aucune duplication de la logique métier, et l'ajout d'un nouveau client resterait simple.

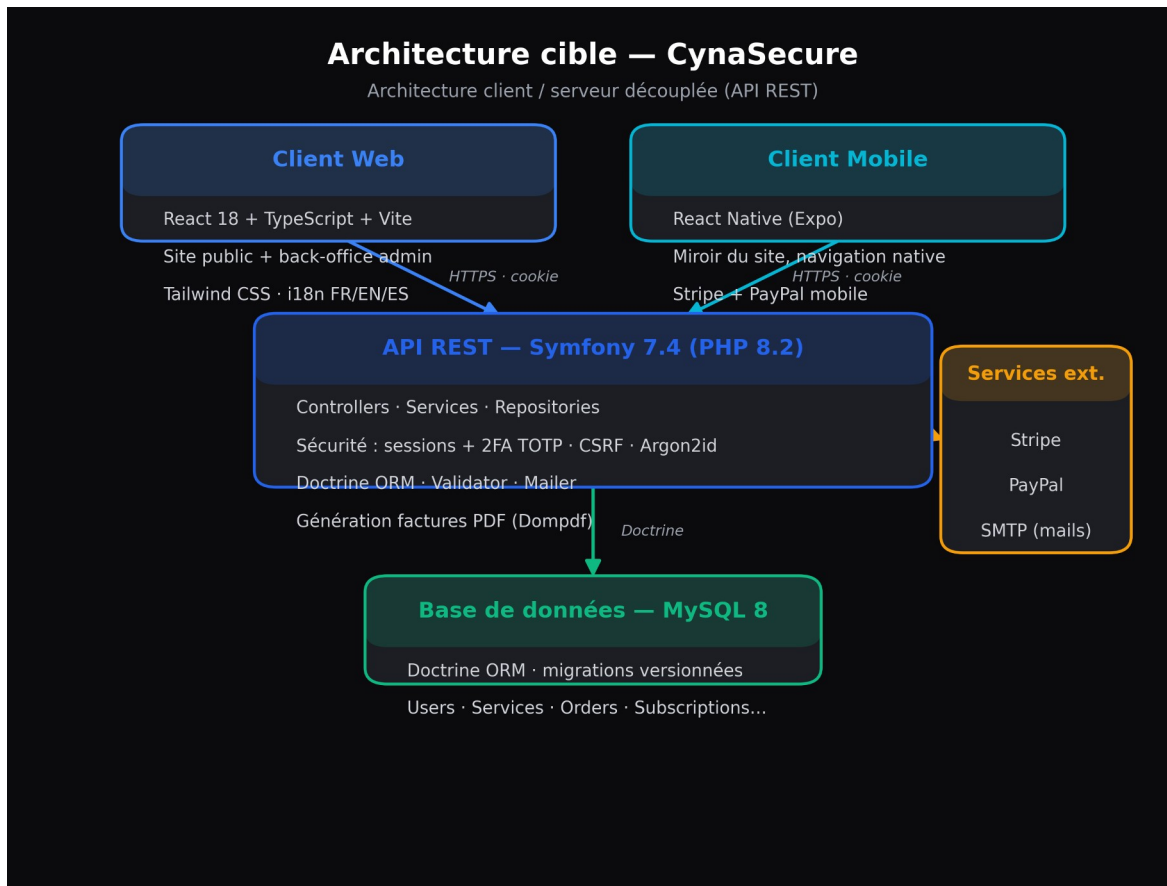


Figure 1 - Architecture client / serveur de CynaSecure

Cette organisation apporte plusieurs garanties :

- Scalabilité : les couches client et serveur évoluent indépendamment, la base MySQL peut être répliquée.
- Sécurité : chaque couche est isolée, les communications sont chiffrées (TLS) et les paiements délégués à des prestataires certifiés.
- Modularité : les services Symfony sont injectables et découplés, ce qui facilite la maintenance.
- Compatibilité mobile : l'application React Native réutilise la même API que le site.
- Robustesse des paiements : validation synchrone à l'écran et confirmation asynchrone par webhooks signés.

## 2. Architecture backend (Symfony)

Le backend suit l'architecture en couches recommandée par Symfony, qui correspond au patron MVC adapté aux API REST. La logique est répartie entre contrôleurs, services et entités, ce qui rend le code lisible et testable.

Organisation effective du code source :

```
backend/
├── config/ Configuration (sécurité, packages, routes)
├── migrations/ Migrations Doctrine versionnées
├── src/
│   ├── Command/ Commandes CLI (ex. renouvellement des abonnements)
│   ├── Controller/ Contrôleurs HTTP (publics + Admin/)
│   ├── Dto/ Objets de transfert (critères de recherche...)
│   ├── Entity/ Entités Doctrine
│   ├── EventSubscriber/ Subscribers (2FA admin...)
│   ├── Repository/ Requêtes Doctrine spécialisées
│   ├── Security/ Handlers d'authentification, 2FA
│   ├── Service/ Logique métier (Stripe, PayPal, factures, fraude...)
│   └── Validator/ Contraintes personnalisées (mot de passe fort)
└── templates/ Gabarits Twig (e-mails, factures PDF)
```

### Contrôleurs (Controller)

Ils reçoivent les requêtes HTTP, appellent les services métier et renvoient les réponses en JSON. Exemples : AuthController, ServiceController, CheckoutController, Admin\AdminStatsController.

### Services (Service)

Ils portent toute la logique métier et sont injectés via le conteneur Symfony. Exemples : StripeService et PayPalService (paiements), InvoiceService (factures PDF), ServiceSearchService (recherche avancée), FraudDetectionService (détection de fraude), ChatbotService (FAQ).

### Entités et repositories

Les entités Doctrine modélisent les tables de la base ; les repositories portent les requêtes spécifiques (recherche, agrégations pour les graphiques). Doctrine gère le mapping objet-relationnel et les migrations.

### Sécurité (Security)

Le pare-feu Symfony protège les routes selon le profil de l'utilisateur. L'authentification repose sur les sessions (cookie), renforcée par une double authentification TOTP pour les comptes administrateurs. Un contrôleur de compte bloque l'accès tant que l'e-mail n'a pas été vérifié.

## 3. Flux de données

Les flux ci-dessous décrivent comment circulent les données entre le client, l'API Symfony, la base MySQL et les services de paiement.

### 3.1. Inscription et vérification de l'e-mail

Étapes :

- L'utilisateur remplit le formulaire d'inscription (validation côté client et côté serveur).
- Le mot de passe est haché en Argon2id avant enregistrement.
- Le compte est créé dans MySQL, un jeton de vérification (haché) est généré, valable 24 h.
- Un e-mail de validation est envoyé via Symfony Mailer.
- À l'ouverture du lien, le compte est activé ; la connexion reste bloquée tant que l'e-mail n'est pas vérifié.

### 3.2. Achat d'un abonnement

Étapes :

- L'utilisateur ajoute un service au panier puis lance le tunnel de commande.
- À l'étape paiement, l'API recalcule le montant côté serveur (jamais celui envoyé par le client).
- Pour Stripe : création d'un PaymentIntent, retour du clientSecret, confirmation côté client avec 3-D Secure.
- L'API vérifie le paiement côté serveur, marque la commande payée, génère la facture et crée les abonnements.
- Un e-mail de confirmation est envoyé à l'utilisateur.

### 3.3. Paiement et webhooks

Le paiement est confirmé à l'écran (synchrone) et validé de façon asynchrone par les webhooks Stripe et PayPal. Cette double validation garantit la cohérence du statut de la commande même si le client perd la connexion juste après le paiement. La signature de chaque webhook est vérifiée côté serveur, et le traitement est idempotent (une commande déjà payée n'est pas retraitée).

### 3.4. Gestion administrateur

Étapes :

- L'administrateur se connecte, puis valide sa double authentification (obligatoire).
- Le pare-feu vérifie le rôle ROLE\_ADMIN avant tout accès aux routes /api/admin.
- Il gère le catalogue, les utilisateurs, les commandes, les abonnements et le contenu du site.
- Les actions sensibles sont tracées dans les journaux applicatifs.

## 4. Modèle de données (ERD)

Le modèle relationnel s'organise autour de trois domaines : le domaine utilisateur (comptes, adresses, moyens de paiement), le domaine commerce (services, catégories, commandes, paiements, abonnements) et le domaine éditorial / support (contenu de la page d'accueil, codes promo, contact, chatbot). Le schéma ci-dessous présente les entités principales et leurs relations.

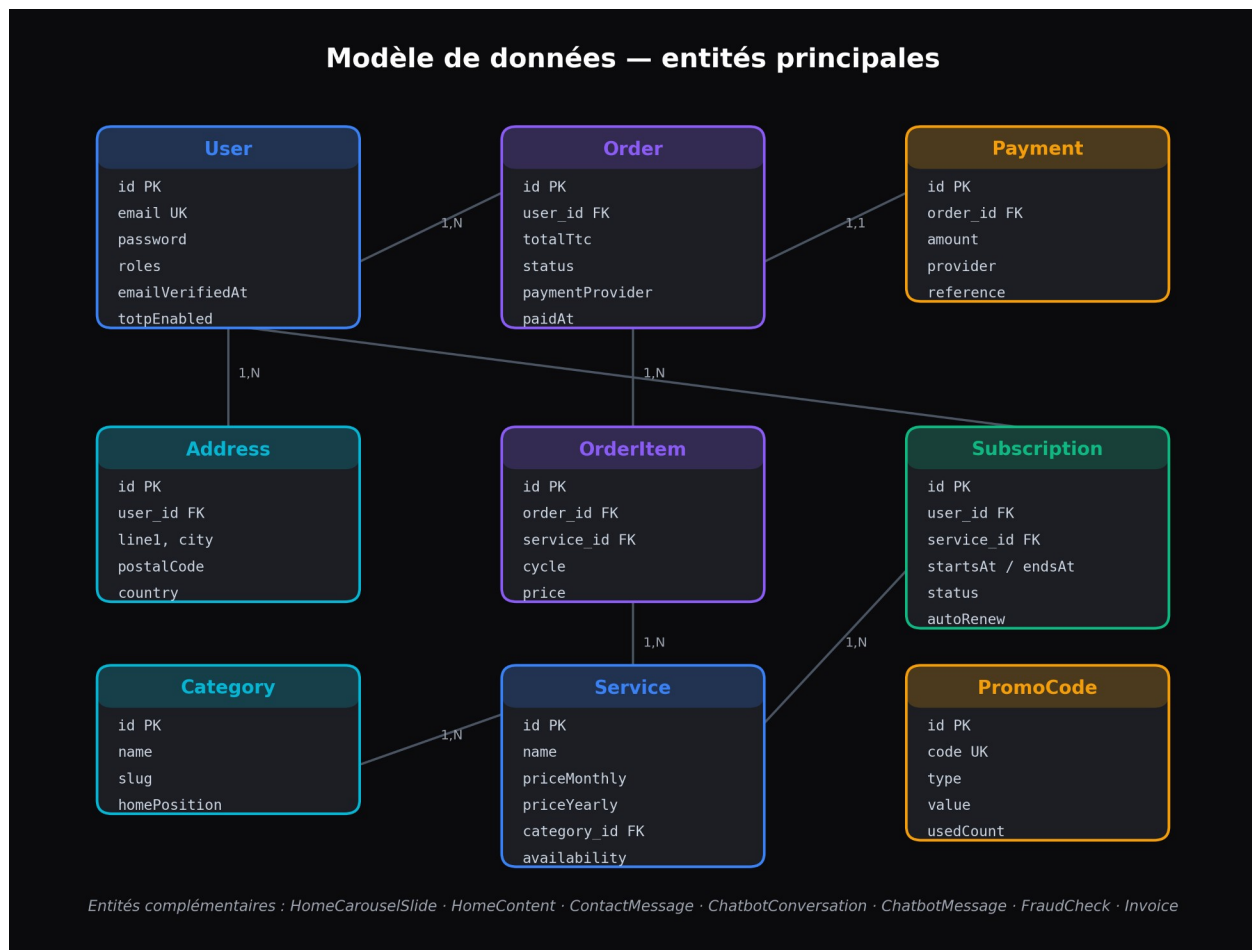


Figure 2 - Modèle entité-relation (entités principales)

Ce modèle couvre l'ensemble des besoins : navigation du catalogue, panier, tunnel de commande, facturation et cycle de vie des abonnements. Il est mis en œuvre avec Doctrine ORM et versionné par des migrations.

## Partie III - Choix technologiques

Cette partie justifie les technologies retenues. Chaque choix répond à un objectif précis : performance, maintenabilité, sécurité ou rapidité de développement.

### 1. Technologies frontend

Le frontend s'appuie sur React, Vite et Tailwind CSS, en TypeScript.

**React (TypeScript)** : bibliothèque à base de composants réutilisables, idéale pour une Single Page Application. Elle gère efficacement les interactions dynamiques du projet : panier, filtres, recherche instantanée, navigation et état de l'utilisateur. Le typage TypeScript fiabilise le code.

**Vite** : outil de build moderne : démarrage instantané du serveur de développement, rechargement à chaud performant et bundle de production optimisé.

**Tailwind CSS** : framework utilitaire qui permet de styliser rapidement et de garantir une cohérence visuelle. Le design est en mode sombre et entièrement responsive (mobile-first).

Le site est multilingue (français, anglais, espagnol) grâce à react-i18next, et conforme au niveau AA des règles d'accessibilité WCAG 2.1.

### 2. Technologies backend

Le backend est développé avec Symfony 7.4 (PHP 8.2), un framework professionnel reconnu pour la robustesse de son écosystème. Le choix de Symfony plutôt qu'un framework léger se justifie par la richesse de ses composants (sécurité, mailer, validation, console) et par la maturité de Doctrine.

Avantages de Symfony pour ce projet :

- Organisation claire et modulaire du code (architecture en couches).
- Sécurité native : gestion de l'authentification, des rôles et des accès.
- Création simple d'API REST : routing, sérialisation JSON, validation, gestion d'erreurs.
- Large écosystème de bundles (2FA, CORS, documentation d'API...).

**Doctrine ORM** : Symfony s'appuie sur Doctrine pour définir les entités, gérer les relations entre les données, écrire des requêtes sûres et versionner le schéma par des migrations. Aucune requête SQL n'est concaténée à la main, ce qui protège contre les injections.

### 3. Base de données

La base de données retenue est MySQL 8, un SGBD relationnel reconnu pour sa fiabilité et sa large compatibilité avec les hébergeurs. Il convient parfaitement aux besoins du projet : transactions ACID pour sécuriser les paiements et les commandes, gestion fine des relations entre utilisateurs, services, commandes et abonnements, et bonnes performances sur les requêtes du catalogue et des statistiques.

L'accès aux données passe exclusivement par Doctrine ORM, qui apporte un mapping objet-relationnel clair, des migrations reproductibles et un typage strict des entités.



## 4. Sécurité et paiement

La sécurité est traitée à tous les niveaux, conformément aux bonnes pratiques de l'OWASP et aux exigences du cahier des charges.

| Domaine                 | Mesure mise en œuvre                                                 |
|-------------------------|----------------------------------------------------------------------|
| Authentification        | Sessions Symfony (cookie HttpOnly, SameSite)                         |
| Double authentification | TOTP, obligatoire pour les administrateurs                           |
| Mots de passe           | Hachage Argon2id, politique forte (8+ car., maj/min/chiffre/spécial) |
| Vérification e-mail     | Jetons aléatoires hachés, valables 24 h                              |
| Rôles                   | ROLE_USER / ROLE_ADMIN, contrôle des routes sensibles                |
| Paiement                | Stripe + PayPal, conformité PCI-DSS, 3-D Secure                      |
| Données bancaires       | Jamais stockées (seul l'identifiant Stripe est conservé)             |
| Webhooks                | Signature vérifiée côté serveur, traitement idempotent               |
| Détection de fraude     | Stripe Radar + scoring interne                                       |
| Attaques web            | Protection CSRF, échappement XSS, requêtes Doctrine paramétrées      |
| Configuration           | Secrets et clés API stockés en variables d'environnement             |

Deux passerelles de paiement sont intégrées, Stripe et PayPal, toutes deux en mode test (sandbox) pendant la phase de développement. Stripe gère les cartes bancaires avec authentification 3-D Secure ; PayPal offre une alternative via le compte de l'utilisateur. Aucune donnée de carte ne transite par nos serveurs.

## 5. Application mobile

L'application mobile est développée en React Native avec Expo, en TypeScript. Elle constitue le miroir du site : elle reprend les mêmes parcours (catalogue, panier, commande, compte, back-office) et consomme la même API Symfony. La navigation est native (React Navigation), le panier et la langue sont persistés localement, et les paiements passent par les SDK mobiles de Stripe et PayPal. L'application partage la charte graphique du site et le support multilingue FR/EN/ES.

## Partie IV - Méthodologie, organisation & planning

### 1. Approche Agile Scrum

Nous avons conduit le projet en méthode Agile, et plus précisément avec le cadre Scrum. Plutôt que de tout planifier en amont, nous avons découpé le travail en incréments courts, ce qui nous a permis d'ajuster régulièrement les priorités et de garder une bonne visibilité sur l'avancement.

Le projet a été fragmenté en sprints d'une semaine, chacun se terminant par un incrément fonctionnel testable. Cette approche a aussi facilité la communication au sein de l'équipe et l'entraide entre les membres.

Les quatre cérémonies Scrum que nous avons pratiquées :

- Planification du sprint : en début de sprint, nous sélectionnons et priorisons les tâches.
- Point quotidien (stand-up) : chaque jour, environ 15 minutes pour partager les avancées et identifier les blocages.
- Démonstration : en fin de sprint, présentation des fonctionnalités développées pour vérifier qu'elles répondent au besoin.
- Rétrospective : bilan du sprint (points positifs, difficultés, actions d'amélioration).

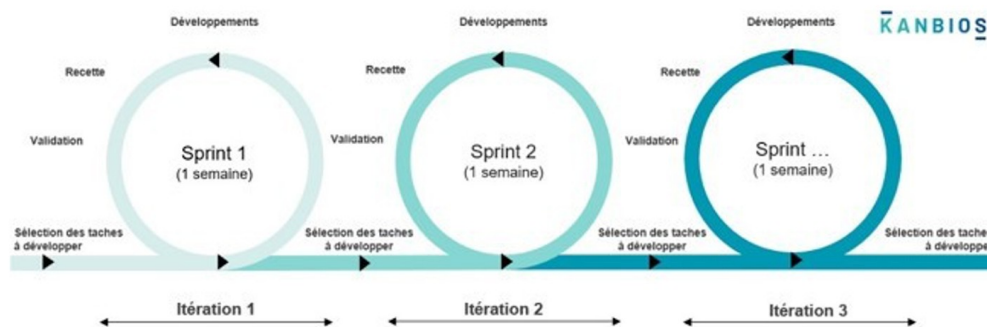


Figure 3 - Cycle itératif des sprints (Agile Scrum)

### Organisation des sprints

Le projet a été découpé en dix sprints. Le tableau ci-dessous décrit, pour chacun, son objectif, ses contraintes, ses dépendances et ses critères de clôture.

| Sprint                  | Objectif                                                                                      | Contrainte                                                         | Dépendance | Critères de fin                                                |
|-------------------------|-----------------------------------------------------------------------------------------------|--------------------------------------------------------------------|------------|----------------------------------------------------------------|
| 1 - Initialisation      | Mettre en place l'environnement (Git, outils, structure), définir les rôles, créer le backlog | Disponibilité des outils ; accord de l'équipe sur les technologies | -          | Dépôt Git fonctionnel ; backlog validé ; environnement prêt    |
| 2 - Analyse des besoins | Recueillir et formaliser les besoins,                                                         | Compréhension correcte du besoin ; temps limité du                 | Sprint 1   | User stories validées ; maquettes réalisées ; backlog priorisé |

|                                         |                                                                                                                           |                                                                    |                |                                                                                                           |
|-----------------------------------------|---------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|----------------|-----------------------------------------------------------------------------------------------------------|
|                                         | rédiger les user stories, premières maquettes                                                                             | cadrage                                                            |                |                                                                                                           |
| <b>3 - Conception technique</b>         | Définir l'architecture, modéliser la base, produire les diagrammes                                                        | Cohérence besoins / technique ; choix technologiques pertinents    | Sprint 2       | Architecture validée ; modèle de données prêt ; diagrammes finalisés                                      |
| <b>4 - Mise en place du backend</b>     | Initialiser l'API Symphony, configurer la base, développer les premières routes                                           | Respect de l'architecture ; bonne gestion des données              | Sprint 3       | API fonctionnelle (CRUD) ; connexion à la base opérationnelle                                             |
| <b>5 - Mise en place du frontend</b>    | Initialiser l'interface, développer les premières pages, intégrer les maquettes                                           | Respect du design ; compatibilité avec le backend                  | Sprints 2 et 4 | Interface affichée correctement ; premières pages fonctionnelles                                          |
| <b>6 - Fonctionnalités principales</b>  | Implémenter les fonctionnalités clés, relier frontend et backend, gérer les interactions                                  | Synchronisation front/back ; gestion des erreurs                   | Sprints 4 et 5 | Fonctionnalités opérationnelles ; données correctement échangées                                          |
| <b>7 - Sécurité et authentification</b> | Mettre en place l'authentification par session + 2FA, gérer les rôles, sécuriser les accès                                | Protection des données ; complexité technique de la sécurité       | Sprint 6       | Authentification fonctionnelle ; accès sécurisé aux routes                                                |
| <b>8 - Tests et amélioration</b>        | Réaliser les tests (unitaires / fonctionnels), corriger les bugs, optimiser                                               | Temps de test limité ; détection complète des erreurs              | Sprints 6 et 7 | Bugs critiques corrigés ; application stable                                                              |
| <b>9 - Déploiement</b>                  | Préparer la mise en production : build optimisé, configuration des variables d'environnement, tests en conditions réelles | Configuration des environnements ; gestion des clés et des secrets | Sprint 8       | Build de production fonctionnel ; application prête à être déployée ; documentation d'installation à jour |
| <b>10 - Finalisation et soutenance</b>  | Finaliser le projet, préparer la démonstration et le support de présentation                                              | Respect des délais ; qualité de la présentation                    | Sprint 9       | Projet finalisé et stable ; soutenance prête                                                              |

## 2. Organisation de l'équipe

L'équipe est composée de quatre personnes : Mouad BOUNOKRA (chef de projet, frontend et base de données), Pharell TITI (backend et API), Nolwen FERNANDES (back-office d'administration) et Wissal LIDAM (application mobile).

Chaque membre occupe un rôle défini, tout en restant polyvalent pour favoriser l'entraide et la montée en compétences collective.

**Chef de projet :** coordination générale, planification (diagramme de Gantt), animation des cérémonies, suivi des livrables, gestion des risques et préparation de la soutenance.

**Développeur frontend :** développement des interfaces avec React et Vite, intégration des données de l'API, design responsive avec Tailwind CSS et soin de l'expérience utilisateur.

**Développeur backend :** conception et développement de l'API REST avec Symfony, modélisation de la base avec Doctrine et MySQL, sécurisation des endpoints et règles métier.

**DevOps / sécurité :** configuration des environnements, mise en place de l'authentification (sessions + 2FA), tests techniques, intégration des paiements et préparation de la mise en production.

Sur le plan de l'organisation interne : deux réunions hebdomadaires de suivi, une démonstration en fin de chaque sprint et une communication continue via Discord.

## 3. Outils de développement

L'équipe s'appuie sur un ensemble d'outils collaboratifs :



**Trello (Kanban)**

gestion des tâches sous forme de User Stories, organisées en colonnes (Backlog, À faire, En cours, À valider, Terminé), avec critères d'acceptation et estimation de charge. Cette organisation permet de prioriser efficacement et de suivre la progression en temps réel.



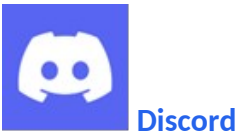
**GitHub**

hébergement du code, suivi de version et revues de code. Le projet est organisé en deux dépôts distincts : un pour le site web et l'API, un pour l'application mobile.



**POSTMAN Postman**

environnement de test d'API utilisé pour exécuter, tester et déboguer les requêtes durant le développement. La documentation de l'API est par ailleurs générée automatiquement au format OpenAPI (Swagger).



communication quotidienne au sein de l'équipe : échanges rapides, partage d'écran pour résoudre les blocages techniques et organisation des réunions à distance.

#### 4. Planning du projet (Gantt)

Le diagramme de Gantt ci-dessous présente le découpage du projet en phases (gestion, conception, développement, qualité, livrables) ainsi que les jalons clés du fil rouge (kick-off, rendez-vous de cadrage, rendu du cadrage, soutenance et livrable final). Il couvre la période d'octobre 2025 à mai 2026.

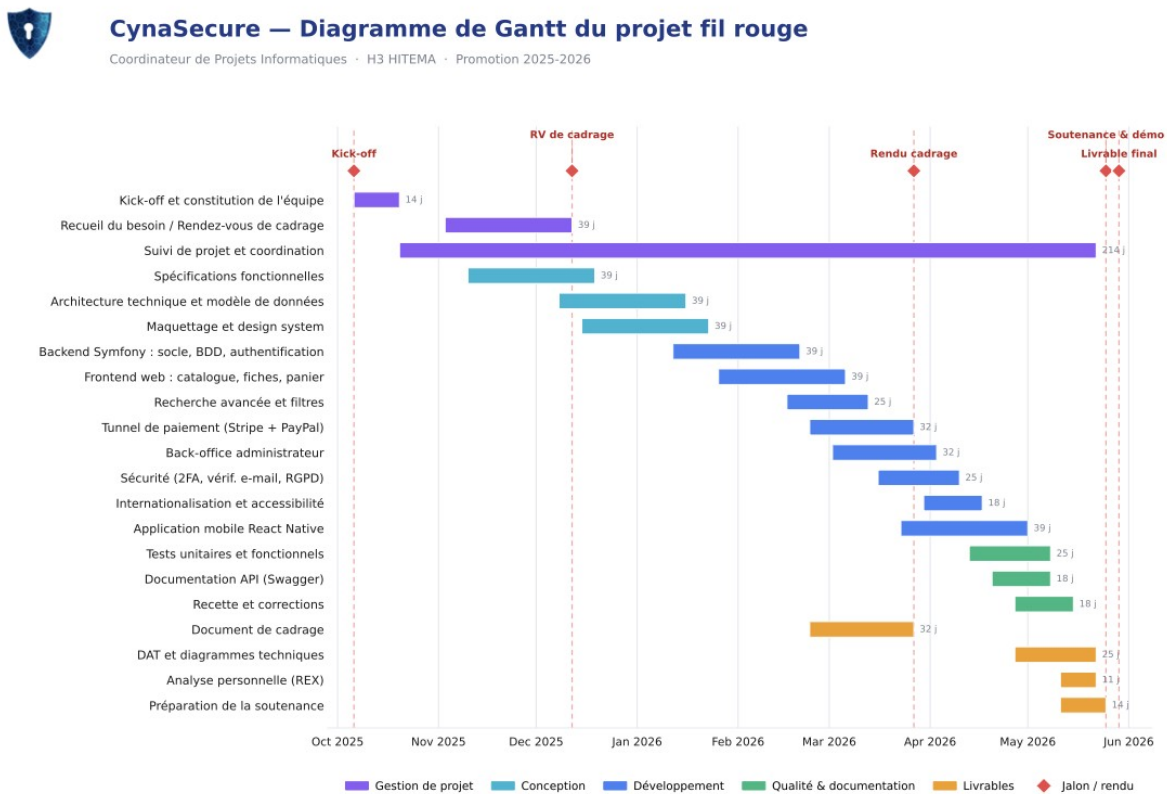


Figure 4 - Diagramme de Gantt du projet

# Annexes

## Annexe A - Glossaire

| Terme    | Définition                                                                                      |
|----------|-------------------------------------------------------------------------------------------------|
| SaaS     | Software as a Service : logiciel fourni en ligne sous forme d'abonnement, sans installation.    |
| SOC      | Security Operations Center : centre de supervision de la sécurité, surveillance 24/7.           |
| EDR      | Endpoint Detection and Response : détection et réponse sur les postes et serveurs.              |
| XDR      | Extended Detection and Response : détection étendue corrélant endpoints, réseau et cloud.       |
| API REST | Interface permettant au frontend et au mobile de communiquer avec le serveur en HTTP.           |
| ORM      | Object-Relational Mapping : couche reliant les objets du code aux tables de la base (Doctrine). |
| ERD      | Entity-Relationship Diagram : schéma des entités de la base et de leurs relations.              |
| TOTP     | Time-based One-Time Password : code temporaire de double authentification (2FA).                |
| PCI-DSS  | Norme de sécurité des données des cartes de paiement.                                           |
| RGPD     | Règlement général sur la protection des données personnelles.                                   |
| MRR      | Monthly Recurring Revenue : revenu mensuel récurrent issu des abonnements.                      |
| Webhook  | Notification automatique envoyée par un service externe (Stripe, PayPal) au serveur.            |
| Sprint   | Itération courte de développement, ici d'une semaine, dans la méthode Scrum.                    |

## Annexe B - Dépôts et environnement

- Dépôt web : code du backend (Symfony) et du frontend (React) + back-office.
- Dépôt mobile : application React Native (Expo).
- Documentation de l'API accessible via Swagger sur la route /api/doc.
- Variables sensibles (clés Stripe/PayPal, base de données) stockées hors du dépôt, dans des fichiers d'environnement.

*Le détail de l'installation et de la configuration figure dans les fichiers README de chaque dépôt et dans le document d'architecture technique (DAT).*